

Two-Pointer Technique

TIP101

Agenda

1 Welcome 🙌

2 UMPIRE

2 Two-Pointer Technique

3 **BREAK**

4 Collaborative Problem Solving

5 Wrap Up

10 minutes

10 minutes

25 minutes

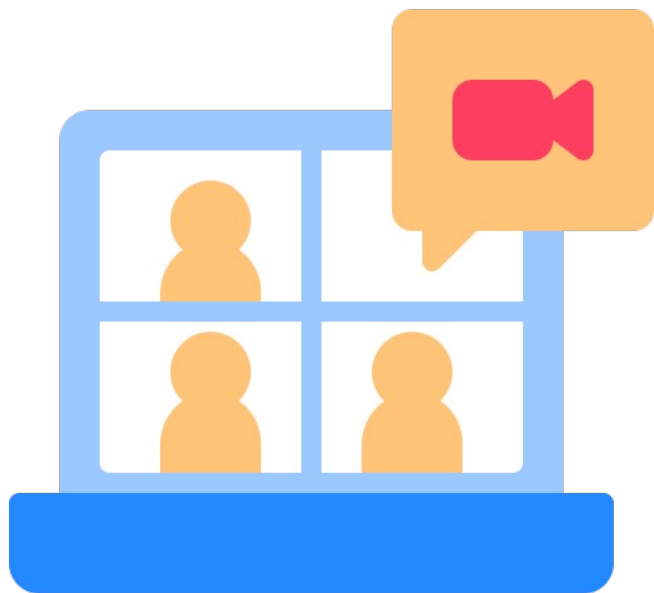
5 minutes

60 minutes

10 minutes

10 minutes

Welcome! 🖐️



**Turn on your
cameras**



**Fill out the entry
ticket**



Icebreaker

5 minutes

Question:

Internships/co-ops can be a great way to experience living in a new city before moving there full time. What would be your dream city for a summer internship?

Answer:

Share your answer in the Zoom chat below. Multiple options are encouraged.

Your Feedback



Glows

- + Got great feedback from our Early Course survey last week
- + Reaching more complex data structures and topics



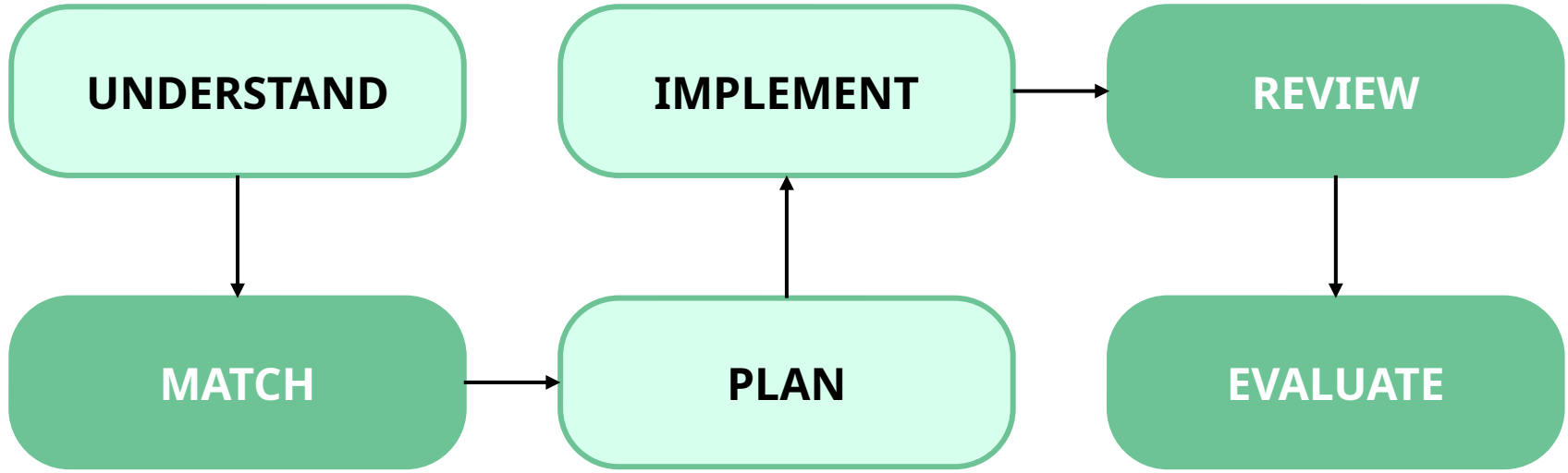
Grows

- Please continue to complete the early course survey if not yet completed last week during class.
- Remember to take turns "navigating" in breakout rooms

10 minutes

UMPIRE

UMPIRE



Match the Problem

- * Identify the appropriate data structure or algorithm
- * Consider where you've seen a similar problem before



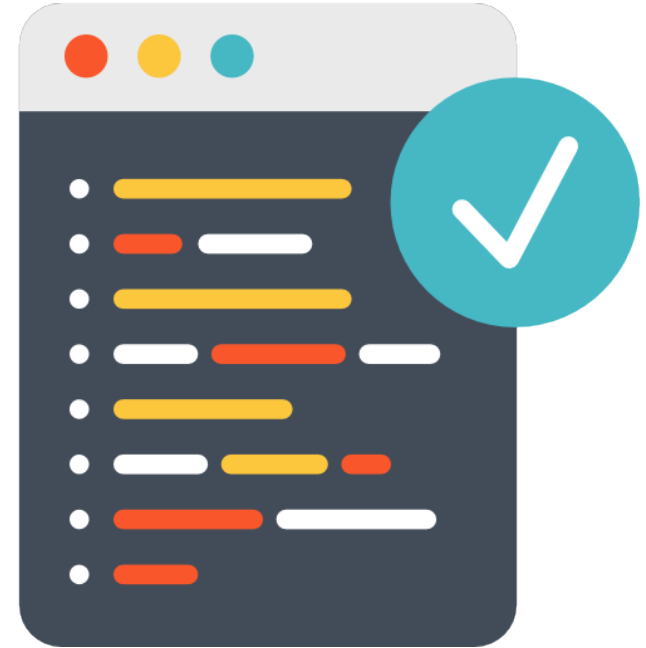
Match the Problem

- * Identify the appropriate data structure or algorithm
- * Consider where you've seen a similar problem before
- * Matching the problem to the right approach is crucial. Are we looking for sums? Are we "searching" or "navigating" through data??
- * Try to think of "simple" versions that you have seen before.



Review the Code

Check for errors, edge cases, and opportunities to optimize.



Review the Code

Check for errors, edge cases, and opportunities to optimize.

- * Once your solution is implemented, reviewing it is essential.
- * Test it with various inputs, especially edge cases like empty arrays or large inputs.



Evaluate the Results

Ensure the solution meets the problem's requirements and handles edge cases



25 minutes

Two-Pointer Technique

Two-Pointer Technique

- * Two pointers move towards or away from each other.
- * Common scenarios:
 - Searching
 - Sorting
 - Comparing elements.
 - Looking for summed local maxima/minima over a window.





Instructor Demo

Given a string `s`, determine if it can become a palindrome by removing **at most one character**.

A palindrome is a word, phrase, or sequence that reads the same backward as forward.

```
def valid_palindrome(s):  
    pass
```

Input: `s = "abca"`

Output: `True`

Break Time!

Take 5 mins to step away from the computer. Feel free to turn off your camera and return promptly after 5 mins.



60 minutes

Collaborative Problem Solving



Breakout Rooms

60 minutes

1. 🏃 Navigate to your breakout room.
2. 🙌 Introduce yourselves!
3. 🧑💻 Work on the problems for your selected unit.

10 minutes

Wrap Up



One Good Thing

Let's reflect on today's session. Maybe you got a question answered, overcame a specific coding challenge, or learned how to do something completely new!

In the chat, share at least one good thing you've learned or experienced during the session today.

Let's celebrate together!



Session Survey Time!





Final Reminders

- ☐ Finish the problem sets for additional practice
- ☐ Take the weekly HackerRank assessment by Sunday @ midnight
- ☐ Visit a **Study Hall Session** to deepen your understanding, work through challenges, and connect with course material alongside your peers!
- ☐ Complete the Early Course Survey if you haven't done so!
 - Check your inbox for your personalized link to complete the survey
- ☐ **Next Session Lookahead:**
 - Big O (Runtime and space complexity analysis).
 - Will look at examples using dictionaries, strings and lists.